

SIMATS ENGINEERING
Department of Computer Science & Engineering
ASSESSMENT TOOL 1- INDUSTRY PROBLEM SOLVING TASK

Course Code:	CSA1223	Course Title:	Computer Architecture
CO Assessed:	CO4	Assessment:	ASSESSMENT TOOL1- INDUSTRY PROBLEM SOLVING TASK
Weightage:	30%	Total Marks:	25
CO4 Statement:	<i>Implement hierarchical memory systems by differentiating cache levels (L1, L2, L3), virtual memory with paging, and advanced memory technologies (DRAM, SRAM, NAND Flash, MRAM, RRAM) based on latency, bandwidth, and throughput characteristics.</i>		
Student Name:	R DINESH KUMAR	Reg.No.:	192312258
Department:	B.E ECE	Date:	24/08/2026

ANNEXURE A
INDUSTRY PROBLEM SOLVING TASK

- 1. Smartphone Performance Optimization a mobile manufacturer observes lag while switching between apps. Analyze the role of L1, L2, L3 cache and DRAM and propose a hierarchical memory redesign to reduce latency and improve throughput.**

Smartphone Performance Optimization

A mobile manufacturer notices that the smartphone becomes slow or lags when the user quickly switches between multiple applications. This problem is mainly related to how frequently the processor needs to access data from different levels of memory. If the required data is not available in a fast cache, the processor has to access slower memory, which increases waiting time.

The **L1 cache** should be the first level of memory used because it is the fastest and is located very close to the processor core. However, its capacity is small. Frequently used instructions and data, such as currently running application code and active variables, should be kept in L1. The **L2 cache** can be made larger than L1 and can store data that does not fit in L1. It provides a balance between speed and capacity. **L3 cache**, being larger but slower than L1 and L2, can act as a shared cache between processor cores and reduce unnecessary accesses to DRAM.

DRAM provides much larger storage capacity than cache, but its latency is considerably higher. When switching between applications, the operating system may need to bring application data from DRAM into the cache. If this happens frequently, the processor spends more time waiting for data, resulting in noticeable lag.

I would therefore propose a hierarchical memory structure such as:

CPU → L1 Cache → L2 Cache → L3 Cache → DRAM

The most frequently accessed data should remain in L1 and L2, while L3 should hold commonly shared or recently used data. DRAM should store the larger working sets of applications. The

operating system can also improve performance by keeping recently used applications and their important data in memory instead of repeatedly loading them from slower storage.

From an engineering point of view, simply increasing cache size is not always the best solution because larger caches consume more chip area and power. A better approach is to optimize each level according to its purpose. L1 should prioritize minimum latency, L2 should provide a balance between latency and capacity, and L3 should provide greater capacity and sharing between cores.

The redesigned hierarchy would reduce the number of DRAM accesses and improve the processor's ability to retrieve frequently needed information quickly. As a result, application switching would become smoother, latency would decrease, and overall system throughput would improve. This approach uses the hierarchical nature of memory rather than depending on one expensive high-speed memory technology for everything. The assessment specifically expects differentiation between L1, L2, L3 and DRAM based on latency, bandwidth and throughput.

2. **Cloud Server Memory Bottleneck**

A cloud service provider experiences high latency in database queries. Compare cache hierarchy vs virtual memory paging and recommend improvements using appropriate memory technologies.

A cloud service provider is experiencing high latency while processing database queries. In a cloud server, database operations can involve a large amount of frequently accessed information. If the required information is not available in cache, the processor may have to access slower levels of memory or use virtual memory paging, increasing query response time.

Cache hierarchy and virtual memory paging solve different problems. Cache memory is designed to reduce processor access latency by keeping frequently used instructions and data close to the CPU. L1 is the fastest but smallest cache, followed by L2 and L3. L3 generally provides larger capacity and can reduce the number of accesses to DRAM.

Virtual memory, on the other hand, allows the operating system to provide processes with a larger logical address space than the available physical DRAM. When required pages are not present in physical memory, a **page fault** occurs and the operating system must retrieve the required page from secondary storage. Compared with cache access, this can introduce a much larger delay.

For a database server, I would prioritize improving the **cache hierarchy and DRAM capacity** before relying heavily on paging. Frequently accessed database indexes, query data and operating-system structures should remain in memory as much as possible. A larger and well-designed L3 cache can reduce accesses to DRAM, while sufficient DRAM capacity can reduce page faults.

The proposed hierarchy would be:

CPU → L1 → L2 → L3 → DRAM → Secondary Storage

The cache levels handle frequently accessed data, while DRAM holds the active database working set. Secondary storage should mainly be used for persistent data and less frequently accessed pages.

Another important improvement would be to monitor cache hit rates, DRAM utilization, page-fault frequency and database query latency. These measurements can identify whether the main problem is poor cache locality, insufficient DRAM or excessive paging. Modern high-performance systems can also use faster memory technologies and optimized storage to reduce the penalty when data must be retrieved from lower levels.

In my view, increasing DRAM alone may not completely solve the problem. If database queries repeatedly access the same data, improving locality and cache utilization can provide a much better result. Similarly, virtual memory should not be treated as a replacement for sufficient physical memory because frequent paging can create a performance bottleneck.

Therefore, the recommended solution is to maintain a strong cache hierarchy, provide sufficient high-bandwidth DRAM for the database workload, and minimize unnecessary virtual-memory paging. This would reduce average memory-access latency and improve database throughput. The assessment specifically asks for comparison between cache hierarchy and virtual-memory paging and appropriate memory technologies.

3. **Autonomous Vehicle Data Processing**

An autonomous vehicle must process sensor data in real time. Evaluate SRAM, DRAM, and MRAM and design a memory hierarchy that maximizes bandwidth and minimizes latency.

An autonomous vehicle continuously receives information from cameras, radar, LiDAR and other sensors. Since the vehicle must make decisions in real time, sensor data cannot remain waiting for a slow memory operation. Even a small delay can affect the speed at which the system detects objects and responds to its surroundings.

For this application, **SRAM, DRAM and MRAM** can be used at different levels of the memory hierarchy. SRAM is very fast and has low latency, making it suitable for frequently accessed real-time data. However, SRAM requires more chip area and is more expensive per bit, so using it for the entire system would not be practical.

DRAM provides much higher capacity and can store larger sensor datasets, intermediate processing results and AI-related data. Its disadvantage is that it has higher latency than SRAM. Therefore, DRAM should be positioned below SRAM in the hierarchy.

MRAM is a non-volatile memory technology that can retain information without continuous power. It can provide fast access while also offering persistence, making it interesting for embedded and automotive applications where power efficiency and reliability are important.

A suitable hierarchy would be:

Processing Unit → SRAM → MRAM → DRAM → Persistent Storage

The fastest-changing and most time-critical sensor information should be placed in SRAM. For example, data currently being used for object detection or collision prediction should have immediate access. MRAM can be used for frequently required persistent information, configuration data and selected workloads where non-volatility is beneficial. DRAM can hold larger sensor buffers, maps and intermediate datasets.

The memory system should also use intelligent data movement. Instead of transferring every sensor reading through the entire hierarchy, the system can identify which data is important for immediate decisions. Critical information can be promoted to SRAM, while older or less urgent information can remain in DRAM.

Bandwidth is also extremely important in an autonomous vehicle because multiple sensors can generate data simultaneously. Therefore, the memory architecture must not only minimize latency but also provide enough bandwidth to prevent data queues from forming.

The proposed design provides a practical balance. SRAM handles **very low-latency access**, DRAM provides **large working capacity and bandwidth**, and MRAM provides **fast non-volatile storage characteristics**. Performance can be evaluated using sensor-processing latency, memory bandwidth, cache/memory hit rate and processor utilization.

Overall, I would not choose a single memory technology for the entire vehicle. A hierarchical design is more realistic because each technology has different strengths. By placing time-critical information in SRAM and larger datasets in DRAM while using MRAM where non-volatility is useful, the vehicle can process sensor information faster and more reliably. This directly addresses the assessment requirement to evaluate SRAM, DRAM and MRAM and design a hierarchy that maximizes bandwidth while minimizing latency.

4. AI Inference Edge Device

An edge AI device needs fast model loading with low power consumption. Compare NAND Flash, DRAM, and RRAM and recommend an optimized hierarchical memory structure.

An edge AI device needs to load and execute AI models quickly while operating under limited power and memory resources. Unlike a large cloud server, an edge device may have a small battery and limited hardware capacity. Therefore, the memory architecture must balance **speed, capacity and power consumption**.

The three important technologies in this problem are **NAND Flash, DRAM and RRAM**. NAND Flash provides high-density non-volatile storage, so it is suitable for permanently storing AI models, application software and other large datasets. However, it is slower than RAM for active processing.

DRAM provides fast working memory and can hold the AI model after it has been loaded from Flash. During inference, the processor can access model parameters and intermediate results from DRAM much faster than from NAND Flash. However, DRAM is volatile and consumes power.

RRAM provides an interesting option because it is a non-volatile memory technology that can offer fast access with potentially lower energy requirements. It can therefore be considered for selected AI workloads where reducing data movement and power consumption is important.

I would design the hierarchy as:

AI Processor → RRAM/fast memory → DRAM → NAND Flash

NAND Flash would act as the permanent model-storage layer. When the device starts, the required AI model would be loaded into DRAM. Frequently accessed parameters or selected inference data could be placed in the faster memory layer. This prevents the processor from repeatedly accessing Flash during inference.

For example, if an edge camera uses an AI model to identify objects, the complete model can remain stored in NAND Flash. When the application starts, the model is transferred into DRAM. The most frequently accessed information can then be kept closer to the AI processor. This reduces model-loading delays and improves inference speed.

Power consumption should also be considered. Keeping unnecessary data in high-speed memory increases energy usage. Therefore, the system should load only the model and data required for the current task. Data that is no longer needed can be removed from the faster memory.

The performance of the design can be evaluated using model-loading time, inference latency, memory bandwidth and energy consumed per inference. If the model-loading time is reduced and the processor spends less time waiting for memory, the device will respond faster while maintaining reasonable power consumption.

Therefore, I would use **NAND Flash for high-capacity permanent storage, DRAM for the main working memory, and RRAM as a fast, low-power non-volatile layer where appropriate**. This gives the edge AI device a practical balance between storage capacity, speed

and energy efficiency. The assessment specifically asks for this comparison and an optimized hierarchical memory structure.

5. High-Performance Gaming Console

A gaming console suffers frame drops during large scene loading. Analyze L3 cache vs DRAM throughput and propose memory hierarchy enhancements.

A gaming console may experience frame drops when loading large game scenes because a large amount of data, including textures, character models, animations and environment information, has to be transferred quickly to the processor and graphics hardware. If the memory system cannot provide data fast enough, the graphics pipeline may wait, resulting in reduced frame rate or stuttering.

L3 cache and DRAM play different roles in this situation. L3 cache is much smaller than DRAM but provides faster access. It is useful for keeping frequently accessed instructions and data close to the processor. DRAM has a much larger capacity and is therefore suitable for storing large game assets, but its access latency is higher.

A gaming console should therefore use a hierarchy such as:

CPU/GPU → L1/L2 → L3 Cache → High-Bandwidth DRAM → Storage

L3 should be used for frequently reused data rather than attempting to store entire game scenes. For example, small pieces of game logic or repeatedly accessed processing data can benefit from cache locality. Large textures and scene assets should remain in DRAM and be transferred efficiently when required.

One important improvement would be to increase the **effective memory bandwidth** available to the graphics processor. Modern games continuously move large amounts of data, so bandwidth can become as important as latency. Faster DRAM and a wider memory interface can allow more scene data to reach the GPU in a shorter time.

Another improvement is intelligent prefetching. The system can predict which assets will be required for the next part of the scene and move them into memory before they are actually requested. This can hide some of the memory-access latency. However, unnecessary prefetching should be avoided because it can waste bandwidth.

The console can also organize game assets according to priority. Frequently required assets can remain readily available in DRAM, while less important assets can be loaded when needed. The storage system should be used to maintain the complete game data, while DRAM acts as the active working area.

Performance can be evaluated using frame time, frame drops, DRAM bandwidth utilization, cache hit rate and scene-loading time. If the memory system can continuously provide the required data to the GPU, the graphics pipeline will spend less time waiting.

Therefore, I would not simply increase L3 cache size. A better solution is to maintain an appropriately sized L3 cache while improving **DRAM bandwidth, data prefetching and memory management**. This provides a better balance between cache latency and the large capacity required by modern games. The assessment specifically asks for analysis of L3 versus DRAM throughput and memory-hierarchy enhancements.

Code of Conduct:

*I **R DINESH KUMAR (192312258)** certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

R DINESH KUMAR

MARKS ALLOCATION

	Understanding of Industry Problem (20%)	Application of Engineering Concepts (25%)	Solution Design (25%)	Use of Tools (15%)	Analysis (10%)
Q1					
Q2					
Q3					
Q4					
Q5					

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Industry Problem	20%	Comprehensive understanding of real-world problem and constraints(1)	Good understanding with minor gaps in requirements	Basic understanding; some requirements unclear	Poor understanding; misinterprets problem context
Application of Engineering Concepts	25%	Applies advanced and relevant concepts effectively to solve the problem(1.25)	Applies appropriate concepts with minor errors	Limited application of concepts	Incorrect or no application of concepts

Solution Design	25%	Innovative, practical, and feasible solution aligned with industry standards(1.25)	Logical and feasible solution with minor gaps	Basic solution with limited feasibility	Unclear or impractical solution
Use of Tools	15%	Effective use of modern tools, technologies, and real data(0.75)	Appropriate use of tools with correct implementation	Limited or inconsistent use of tools	Minimal or incorrect use of tools
Analysis	10%	Thorough analysis with validated results and performance evaluation(0.5)	Good analysis with reasonable validation	Basic analysis with limited validation	Poor or no analysis

SIMATS ENGINEERING
Department of Computer Science & Engineering
ASSESSMENT TOOL 1- INDUSTRY PROBLEM SOLVING TASK

Course Code:	CSA1223	Course Title:	Computer Architecture
CO Assessed:	CO4	Assessment:	ASSESSMENT TOOL1- INDUSTRY PROBLEM SOLVING TASK
Weightage:	30%	Total Marks:	25
CO4 Statement:	<i>Implement hierarchical memory systems by differentiating cache levels (L1, L2, L3), virtual memory with paging, and advanced memory technologies (DRAM, SRAM, NAND Flash, MRAM, RRAM) based on latency, bandwidth, and throughput characteristics.</i>		
Student Name:	R DINESH KUMAR	Reg.No.:	192312258
Department:	B.E ECE	Date:	24/08/2026

ANNEXURE A
INDUSTRY PROBLEM SOLVING TASK

- 1. Smartphone Performance Optimization a mobile manufacturer observes lag while switching between apps. Analyze the role of L1, L2, L3 cache and DRAM and propose a hierarchical memory redesign to reduce latency and improve throughput.**

Smartphone Performance Optimization

A mobile manufacturer notices that the smartphone becomes slow or lags when the user quickly switches between multiple applications. This problem is mainly related to how frequently the processor needs to access data from different levels of memory. If the required data is not available in a fast cache, the processor has to access slower memory, which increases waiting time.

The **L1 cache** should be the first level of memory used because it is the fastest and is located very close to the processor core. However, its capacity is small. Frequently used instructions and data, such as currently running application code and active variables, should be kept in L1. The **L2 cache** can be made larger than L1 and can store data that does not fit in L1. It provides a balance between speed and capacity. **L3 cache**, being larger but slower than L1 and L2, can act as a shared cache between processor cores and reduce unnecessary accesses to DRAM.

DRAM provides much larger storage capacity than cache, but its latency is considerably higher. When switching between applications, the operating system may need to bring application data from DRAM into the cache. If this happens frequently, the processor spends more time waiting for data, resulting in noticeable lag.

I would therefore propose a hierarchical memory structure such as:

CPU → L1 Cache → L2 Cache → L3 Cache → DRAM

The most frequently accessed data should remain in L1 and L2, while L3 should hold commonly shared or recently used data. DRAM should store the larger working sets of applications. The

operating system can also improve performance by keeping recently used applications and their important data in memory instead of repeatedly loading them from slower storage.

From an engineering point of view, simply increasing cache size is not always the best solution because larger caches consume more chip area and power. A better approach is to optimize each level according to its purpose. L1 should prioritize minimum latency, L2 should provide a balance between latency and capacity, and L3 should provide greater capacity and sharing between cores.

The redesigned hierarchy would reduce the number of DRAM accesses and improve the processor's ability to retrieve frequently needed information quickly. As a result, application switching would become smoother, latency would decrease, and overall system throughput would improve. This approach uses the hierarchical nature of memory rather than depending on one expensive high-speed memory technology for everything. The assessment specifically expects differentiation between L1, L2, L3 and DRAM based on latency, bandwidth and throughput.

2. **Cloud Server Memory Bottleneck**

A cloud service provider experiences high latency in database queries. Compare cache hierarchy vs virtual memory paging and recommend improvements using appropriate memory technologies.

A cloud service provider is experiencing high latency while processing database queries. In a cloud server, database operations can involve a large amount of frequently accessed information. If the required information is not available in cache, the processor may have to access slower levels of memory or use virtual memory paging, increasing query response time.

Cache hierarchy and virtual memory paging solve different problems. Cache memory is designed to reduce processor access latency by keeping frequently used instructions and data close to the CPU. L1 is the fastest but smallest cache, followed by L2 and L3. L3 generally provides larger capacity and can reduce the number of accesses to DRAM.

Virtual memory, on the other hand, allows the operating system to provide processes with a larger logical address space than the available physical DRAM. When required pages are not present in physical memory, a **page fault** occurs and the operating system must retrieve the required page from secondary storage. Compared with cache access, this can introduce a much larger delay.

For a database server, I would prioritize improving the **cache hierarchy and DRAM capacity** before relying heavily on paging. Frequently accessed database indexes, query data and operating-system structures should remain in memory as much as possible. A larger and well-designed L3 cache can reduce accesses to DRAM, while sufficient DRAM capacity can reduce page faults.

The proposed hierarchy would be:

CPU → L1 → L2 → L3 → DRAM → Secondary Storage

The cache levels handle frequently accessed data, while DRAM holds the active database working set. Secondary storage should mainly be used for persistent data and less frequently accessed pages.

Another important improvement would be to monitor cache hit rates, DRAM utilization, page-fault frequency and database query latency. These measurements can identify whether the main problem is poor cache locality, insufficient DRAM or excessive paging. Modern high-performance systems can also use faster memory technologies and optimized storage to reduce the penalty when data must be retrieved from lower levels.

In my view, increasing DRAM alone may not completely solve the problem. If database queries repeatedly access the same data, improving locality and cache utilization can provide a much better result. Similarly, virtual memory should not be treated as a replacement for sufficient physical memory because frequent paging can create a performance bottleneck.

Therefore, the recommended solution is to maintain a strong cache hierarchy, provide sufficient high-bandwidth DRAM for the database workload, and minimize unnecessary virtual-memory paging. This would reduce average memory-access latency and improve database throughput. The assessment specifically asks for comparison between cache hierarchy and virtual-memory paging and appropriate memory technologies.

3. **Autonomous Vehicle Data Processing**

An autonomous vehicle must process sensor data in real time. Evaluate SRAM, DRAM, and MRAM and design a memory hierarchy that maximizes bandwidth and minimizes latency.

An autonomous vehicle continuously receives information from cameras, radar, LiDAR and other sensors. Since the vehicle must make decisions in real time, sensor data cannot remain waiting for a slow memory operation. Even a small delay can affect the speed at which the system detects objects and responds to its surroundings.

For this application, **SRAM, DRAM and MRAM** can be used at different levels of the memory hierarchy. SRAM is very fast and has low latency, making it suitable for frequently accessed real-time data. However, SRAM requires more chip area and is more expensive per bit, so using it for the entire system would not be practical.

DRAM provides much higher capacity and can store larger sensor datasets, intermediate processing results and AI-related data. Its disadvantage is that it has higher latency than SRAM. Therefore, DRAM should be positioned below SRAM in the hierarchy.

MRAM is a non-volatile memory technology that can retain information without continuous power. It can provide fast access while also offering persistence, making it interesting for embedded and automotive applications where power efficiency and reliability are important.

A suitable hierarchy would be:

Processing Unit → SRAM → MRAM → DRAM → Persistent Storage

The fastest-changing and most time-critical sensor information should be placed in SRAM. For example, data currently being used for object detection or collision prediction should have immediate access. MRAM can be used for frequently required persistent information, configuration data and selected workloads where non-volatility is beneficial. DRAM can hold larger sensor buffers, maps and intermediate datasets.

The memory system should also use intelligent data movement. Instead of transferring every sensor reading through the entire hierarchy, the system can identify which data is important for immediate decisions. Critical information can be promoted to SRAM, while older or less urgent information can remain in DRAM.

Bandwidth is also extremely important in an autonomous vehicle because multiple sensors can generate data simultaneously. Therefore, the memory architecture must not only minimize latency but also provide enough bandwidth to prevent data queues from forming.

The proposed design provides a practical balance. SRAM handles **very low-latency access**, DRAM provides **large working capacity and bandwidth**, and MRAM provides **fast non-volatile storage characteristics**. Performance can be evaluated using sensor-processing latency, memory bandwidth, cache/memory hit rate and processor utilization.

Overall, I would not choose a single memory technology for the entire vehicle. A hierarchical design is more realistic because each technology has different strengths. By placing time-critical information in SRAM and larger datasets in DRAM while using MRAM where non-volatility is useful, the vehicle can process sensor information faster and more reliably. This directly addresses the assessment requirement to evaluate SRAM, DRAM and MRAM and design a hierarchy that maximizes bandwidth while minimizing latency.

4. AI Inference Edge Device

An edge AI device needs fast model loading with low power consumption. Compare NAND Flash, DRAM, and RRAM and recommend an optimized hierarchical memory structure.

An edge AI device needs to load and execute AI models quickly while operating under limited power and memory resources. Unlike a large cloud server, an edge device may have a small battery and limited hardware capacity. Therefore, the memory architecture must balance **speed, capacity and power consumption**.

The three important technologies in this problem are **NAND Flash, DRAM and RRAM**. NAND Flash provides high-density non-volatile storage, so it is suitable for permanently storing AI models, application software and other large datasets. However, it is slower than RAM for active processing.

DRAM provides fast working memory and can hold the AI model after it has been loaded from Flash. During inference, the processor can access model parameters and intermediate results from DRAM much faster than from NAND Flash. However, DRAM is volatile and consumes power.

RRAM provides an interesting option because it is a non-volatile memory technology that can offer fast access with potentially lower energy requirements. It can therefore be considered for selected AI workloads where reducing data movement and power consumption is important.

I would design the hierarchy as:

AI Processor → RRAM/fast memory → DRAM → NAND Flash

NAND Flash would act as the permanent model-storage layer. When the device starts, the required AI model would be loaded into DRAM. Frequently accessed parameters or selected inference data could be placed in the faster memory layer. This prevents the processor from repeatedly accessing Flash during inference.

For example, if an edge camera uses an AI model to identify objects, the complete model can remain stored in NAND Flash. When the application starts, the model is transferred into DRAM. The most frequently accessed information can then be kept closer to the AI processor. This reduces model-loading delays and improves inference speed.

Power consumption should also be considered. Keeping unnecessary data in high-speed memory increases energy usage. Therefore, the system should load only the model and data required for the current task. Data that is no longer needed can be removed from the faster memory.

The performance of the design can be evaluated using model-loading time, inference latency, memory bandwidth and energy consumed per inference. If the model-loading time is reduced and the processor spends less time waiting for memory, the device will respond faster while maintaining reasonable power consumption.

Therefore, I would use **NAND Flash for high-capacity permanent storage, DRAM for the main working memory, and RRAM as a fast, low-power non-volatile layer where appropriate**. This gives the edge AI device a practical balance between storage capacity, speed

and energy efficiency. The assessment specifically asks for this comparison and an optimized hierarchical memory structure.

5. High-Performance Gaming Console

A gaming console suffers frame drops during large scene loading. Analyze L3 cache vs DRAM throughput and propose memory hierarchy enhancements.

A gaming console may experience frame drops when loading large game scenes because a large amount of data, including textures, character models, animations and environment information, has to be transferred quickly to the processor and graphics hardware. If the memory system cannot provide data fast enough, the graphics pipeline may wait, resulting in reduced frame rate or stuttering.

L3 cache and DRAM play different roles in this situation. L3 cache is much smaller than DRAM but provides faster access. It is useful for keeping frequently accessed instructions and data close to the processor. DRAM has a much larger capacity and is therefore suitable for storing large game assets, but its access latency is higher.

A gaming console should therefore use a hierarchy such as:

CPU/GPU → L1/L2 → L3 Cache → High-Bandwidth DRAM → Storage

L3 should be used for frequently reused data rather than attempting to store entire game scenes. For example, small pieces of game logic or repeatedly accessed processing data can benefit from cache locality. Large textures and scene assets should remain in DRAM and be transferred efficiently when required.

One important improvement would be to increase the **effective memory bandwidth** available to the graphics processor. Modern games continuously move large amounts of data, so bandwidth can become as important as latency. Faster DRAM and a wider memory interface can allow more scene data to reach the GPU in a shorter time.

Another improvement is intelligent prefetching. The system can predict which assets will be required for the next part of the scene and move them into memory before they are actually requested. This can hide some of the memory-access latency. However, unnecessary prefetching should be avoided because it can waste bandwidth.

The console can also organize game assets according to priority. Frequently required assets can remain readily available in DRAM, while less important assets can be loaded when needed. The storage system should be used to maintain the complete game data, while DRAM acts as the active working area.

Performance can be evaluated using frame time, frame drops, DRAM bandwidth utilization, cache hit rate and scene-loading time. If the memory system can continuously provide the required data to the GPU, the graphics pipeline will spend less time waiting.

Therefore, I would not simply increase L3 cache size. A better solution is to maintain an appropriately sized L3 cache while improving **DRAM bandwidth, data prefetching and memory management**. This provides a better balance between cache latency and the large capacity required by modern games. The assessment specifically asks for analysis of L3 versus DRAM throughput and memory-hierarchy enhancements.

Code of Conduct:

*I **R DINESH KUMAR (192312258)** certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

R DINESH KUMAR

MARKS ALLOCATION

	Understanding of Industry Problem (20%)	Application of Engineering Concepts (25%)	Solution Design (25%)	Use of Tools (15%)	Analysis (10%)
Q1					
Q2					
Q3					
Q4					
Q5					

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Industry Problem	20%	Comprehensive understanding of real-world problem and constraints(1)	Good understanding with minor gaps in requirements	Basic understanding; some requirements unclear	Poor understanding; misinterprets problem context
Application of Engineering Concepts	25%	Applies advanced and relevant concepts effectively to solve the problem(1.25)	Applies appropriate concepts with minor errors	Limited application of concepts	Incorrect or no application of concepts

Solution Design	25%	Innovative, practical, and feasible solution aligned with industry standards(1.25)	Logical and feasible solution with minor gaps	Basic solution with limited feasibility	Unclear or impractical solution
Use of Tools	15%	Effective use of modern tools, technologies, and real data(0.75)	Appropriate use of tools with correct implementation	Limited or inconsistent use of tools	Minimal or incorrect use of tools
Analysis	10%	Thorough analysis with validated results and performance evaluation(0.5)	Good analysis with reasonable validation	Basic analysis with limited validation	Poor or no analysis